Anonymizing and Encrypting Using Python

Explore how to anonymize and encrypt sensitive data using Python during the transform phase of ETL pipelines. Understand methods like hashing, masking, perturbation, and both symmetric and asymmetric encryption to secure personal and confidential information effectively.

When dealing with sensitive data such as passwords, financial data, medical records, or confidential business information, we often need to protect it somehow. During the transform stage of the ETL pipeline, we might need to employ data anonymization or data encryption methods.

Data anonymization

During **data anonymization,** we remove or obscure *Personally Identifiable Information(PII)* from a dataset to keep the privacy of users and clients.

There are several methods of anonymizing data, including:

- **Masking**: Replacing sensitive information with characters such as asterisks.
- **Perturbation**: Adding random noise or error to the data to obscure specific values. For example, a dataset of GPS locations of users used for a statistical analysis might be perturbed by adding some random, normally distributed noise to keep the exact coordinates hidden while still allowing the analysts to perform statistical analysis on the overall distribution of the dataset.

Ask

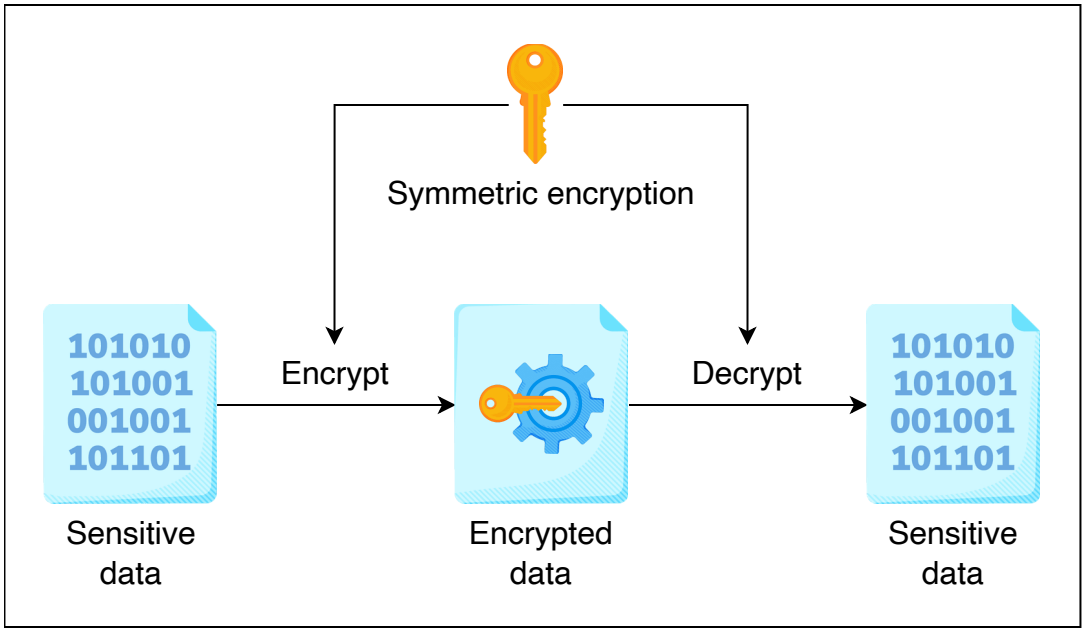
- **Hashing**: Using a hash function to transform the original data into a fixed-size and unique string of characters. For example, when storing password data, we hash the passwords and compare the hash to the stored hash value during authentication.

Data encryption

Data encryption is the process of converting sensitive data into an encrypted form to protect it from unauthorized access. Only the users who encrypted the data in the first place can decipher and access it.

There are two types of encryption methods:

1. **Symmetric encryption:** In symmetric encryption, we use the same key or encryption function to both encrypt the data and decipher it. This means that symmetric encryption is only useful when the key is kept confidential and protected from unauthorized access.



Symmetric encryption

Ask

2. **Asymmetric encryption:** In this encryption, a public key is used to encrypt the data, while a different key, called a private key, is used to decipher it. This

lowers the chances of unauthorized users deciphering the data.

Asymmetric encryption

In general, asymmetric encryption is more secure than symmetric encryption but is also slower and less suited for large volumes of data.

We can use Python encryption libraries such as `pycrypto` and `cryptograph` to implement data encryption for sensitive data.

Example

Suppose a healthcare organization collects patient data from various sources for analysis and reporting. They want to ensure that patient data is protected from unauthorized access and that it complies with regulations such as HIPAA.

> **Note:** The Health Insurance Portability and Accountability Act of 1996 (HIPAA) is a federal law that required the creation of national standards to protect sensitive patient health information from being disclosed without the patient's consent or knowledge.

Ask

To comply with HIPAA regulations, we must protect patients' protected health information (PHI) and electronic PHI (ePHI) when the data is at rest. To meet these requirements, the organization must implement data anonymization and data encrypting before the data reaches the destination repository.

The following is a sample of a single patient data:

main.py Python 3.10.4

```
# Sample patient data
patient_data = {
    'patient_id': '1234',
    'first_name': 'Sherlock',
    'last_name': 'Holmes',
    'address': '221B Baker Street, London',
    'date_of_birth': '06/01/1854',
    'diagnosis': 'Flu',
    'treatment': 'Rest and Hydration',
    'medication': 'Aspirin'
}
```

Sample patient data

Data anonymization

First, let's anonymize the data by hashing it. We'll use Python's library `hashlib` to apply a hash function on the personally identifiable columns of the patient, which are the `patient_id`, `first_name`, `last_name`, `address`, and `date_of_birth`.

main.py Python 3.10.4

```
import hashlib

# Sample patient data
patient_data = {
```

Ask

```
        'patient_id': '1234',
        'first_name': 'Sherlock',
        'last_name': 'Holmes',
        'address': '221B Baker Street, London',
        'date_of_birth': '06/01/1854',
        'diagnosis': 'Flu',
        'treatment': 'Rest and Hydration',
        'medication': 'Aspirin'
}
```

Run

Hashing personally identifiable patient data

- **Line 17:** We create an empty dictionary to store the anonymized data.
- **Lines 18–22:** We encode and convert the column into a sequence of bytes, then apply a hash function to anonymize the data.

Data encryption

After hashing columns that can risk the patient's privacy, we should encrypt columns with sensitive medical data. We'll use Python's library `cryptography` to apply symmetric encryption on `patient_id`, `diagnosis`, `treatment`, and `medication` columns.

Click the "Run" button in the following script to generate a symmetric key to encrypt and decipher data, then encrypt sensitive information using that key.

main.py Python 3.10.4

```
import hashlib
from cryptography.fernet import Fernet

# Sample patient data
patient_data = {
    'patient_id': '1234',
    'first_name': 'Sherlock',
    'last_name': 'Holmes',
    'address': '221B Baker Street, London',
    'date_of_birth': '06/01/1854',
```

Ask

```
        'diagnosis': 'Flu',
        'treatment': 'Rest and Hydration',
```

Run

Encrypting sensitive data

- **Line 17:** We generate a key to be used for encryption and decryption.
- **Lines 22–26:** We encode and convert the column into a sequence of bytes, then encrypt the data using the generated key.

When needed, we can decipher the data by using the same key we used to encrypt it. Click the “Run” button in the following script to decipher the encrypted data.

main.py Python 3.10.4

```
import hashlib
from cryptography.fernet import Fernet

# Sample patient data
patient_data = {
    'patient_id': '1234',
    'first_name': 'Sherlock',
    'last_name': 'Holmes',
    'address': '221B Baker Street, London',
    'date_of_birth': '06/01/1854',
    'diagnosis': 'Flu',
    'treatment': 'Rest and Hydration',
    'medication': 'Aspirin'
```

Run

Decrypting sensitive data

- **Line 17:** We generate a key to be used for encryption and decryption.
- **Lines 35–38:** We decrypt the data into a sequence of bytes, then con bytes back to the original data.

Ask